Concepts & Theory

1. Cubic Speed–Power Law (Admiralty Law)

A ship's propulsive power — and therefore its fuel burn rate — grows roughly with the cube of its speed through the water. This is a fundamental result from naval architecture: water resistance has a viscous component (proportional to speed squared) and a wave-making component (proportional to speed cubed at typical Froude numbers). The practical implication is dramatic: doubling speed increases fuel burn rate roughly 8× ($2^3 = 8$). This is why even a small speed reduction yields large fuel savings.

```
fuel_rate ∝ displacement^(2/3) × speed³
```

2. Displacement

Displacement is the mass of water a ship pushes aside, equal to the ship's total mass (hull + cargo + fuel + stores). It is measured in tonnes. A heavier ship sits deeper in the water, has more wetted surface area, and therefore experiences more drag. In the Admiralty formula, fuel rate scales with displacement^(2/3), reflecting the relationship between a ship's mass and its wetted surface area. Our three Indian Coast Guard ship classes have very different displacements: Interceptor (~60 t), Fast Patrol (~550 t), Offshore Patrol (~2000 t).

3. Hull Coefficient

The hull coefficient (block coefficient, Cb) describes how 'full' or 'slim' a hull shape is. It is the ratio of the ship's underwater volume to the volume of the smallest rectangular box that could enclose it. A value near 1.0 means a boxy hull (like a tanker, more drag), while a value near 0.4 means a slim, hydrodynamic hull (like a patrol vessel, less drag). In our model, it acts as a direct multiplier on fuel rate — a higher coefficient means more resistance.

4. Hotel / Auxiliary Load

Even when a ship is stationary (zero speed), it still burns fuel. Generators must run to power navigation systems, radar, communications, lighting, air conditioning, and crew facilities. This speed-independent fuel consumption is called the hotel load or auxiliary load. In our model:

hotel_load = 0.02 × displacement^(2/3)

This is critical because it makes the total-voyage-fuel curve U-shaped: at very low speeds the

ship takes so long that hotel load accumulates into a large total, even though the propulsive burn rate is low.

5. Slow Steaming

Slow steaming is the real-world practice of deliberately sailing below a ship's design speed to save fuel. Because fuel rate scales with speed³ but voyage time only scales with 1/speed, total voyage fuel = rate(speed) × time = rate(speed) × distance / speed has a minimum at some 'economic speed' that is well below the ship's maximum. The constraint is operational: you must arrive by a deadline. Our optimizer finds the slowest feasible speed that meets the deadline — this is exactly the slow-steaming decision a fleet operator makes.

```
total_fuel = fuel_rate(speed) × distance / (24 × speed)
```

6. The U-Shaped Fuel Curve

Total voyage fuel is NOT simply 'slower = less fuel'. Two competing effects create a U-shape:

- Going faster → higher fuel rate (cubic law) → more fuel
- Going slower → longer voyage time → hotel load accumulates → more fuel

The bottom of the U is the 'economic speed'. Under a tight deadline, the minimum feasible speed may be above the economic speed (you're forced to burn more). Under a relaxed deadline, the optimizer can reach the true economic speed — and going even slower would actually waste fuel.

7. Environmental Factors

A ship's actual fuel consumption depends heavily on conditions at sea:

- Wave height (m): Higher waves increase hull resistance. Our model uses a multiplicative factor (1 + 0.06 × wave_height) on propulsive fuel.

- Wind speed (knots): Headwinds add aerodynamic drag on the superstructure. Wind correlates with sea state (bigger waves usually mean stronger wind). Our factor: (1 + 0.005 × wind_speed).

- Ocean current (knots): A favourable current reduces speed-through-water (the ship moves over ground faster than it moves through the water). An opposing current does the opposite. We model this as effective_speed = speed_over_ground − current.

- Load condition: A fully laden ship (more cargo/stores) displaces more water, increasing drag. We model 'Laden' as a 1.15× factor vs 'Ballast' at 0.85×.

8. IMO Emission Factors

The International Maritime Organization (IMO) publishes standard CO_2 emission factors for different marine fuels, measured in tonnes of CO_2 per tonne of fuel burned:

- MGO (Marine Gas Oil): 3.206 t CO_2/t fuel
- VLSFO (Very Low Sulphur): 3.151 t CO_2/t fuel
- HFO (Heavy Fuel Oil): 3.114 t CO_2/t fuel

These are official values used in IMO's Carbon Intensity Indicator (CII) calculations. Our project uses them to convert fuel savings into CO_2 savings, which matters for environmental compliance reporting.

9. Synthetic Data Generation

Since we don't have access to real Indian Coast Guard operational data, we generate synthetic training data using the physics described above. The generator (generate_data.py) creates 10,000 voyage records by:

1. Sampling a ship class (Interceptor / Fast Patrol / Offshore Patrol)
2. Drawing ship parameters from Gaussian distributions around each class's typical specs (displacement, engine power, hull coefficient)
3. Sampling random sea conditions (wave height from a Gamma distribution, correlated wind, random current)
4. Computing the true fuel rate from physics + adding 6% multiplicative noise to simulate real-world measurement uncertainty

The key insight: the ML model must rediscover the cubic speed–power law from this noisy data — which it does ($R^2 \approx 0.99$).

10. Feature Engineering: Static vs Dynamic Features

We separate features into two categories:

Static features (don't change during a voyage):

- displacement_tonnes — ship's mass
- engine_power_kw — installed engine power
- hull_coefficient — hull shape factor
- ship_type — categorical (Interceptor / Fast_Patrol / Offshore_Patrol)
- fuel_type — categorical (MGO / VLSFO / HFO)

Dynamic features (change during a voyage):

- speed_knots — the decision variable

- wave_height_m — sea state
- wind_speed_kn — wind conditions
- current_speed_kn — ocean current
- load_condition — categorical (Laden / Ballast)

This separation is important: the optimizer changes the dynamic feature (speed) while holding the static features fixed for a given ship.

11. ML Models: Linear Regression, Random Forest, Gradient Boosting

We train three models to predict fuel rate (tonnes/day) from the features above:

Linear Regression (baseline): Fits a straight-line relationship. It fails badly ($R^2 = 0.65$) because the true relationship is cubic — a linear model cannot capture fuel_rate $\propto$ speed^3 across ships spanning 5–90 t/day.

Random Forest (200 trees): An ensemble of decision trees, each trained on a random subset of data and features. It captures the nonlinear physics well ($R^2 = 0.98$) because decision trees can learn arbitrary nonlinear splits.

Gradient Boosting (HistGradientBoosting): Builds trees sequentially, where each new tree corrects the errors of the previous ensemble. It achieves the best accuracy ($R^2 = 0.987$) and is saved as the production model.

The linear model's failure is not a weakness — it's a demonstration of WHY nonlinear ML is needed for this problem.

12. ColumnTransformer and One-Hot Encoding (for ML)

Our dataset has both numeric features (speed, displacement) and categorical features (ship_type, fuel_type, load_condition). ML models need numbers, not strings. One-hot encoding converts a category with K values into K binary columns — e.g., ship_type becomes three columns: is_Interceptor, is_Fast_Patrol, is_Offshore_Patrol, exactly one of which is 1.

Scikit-learn's ColumnTransformer lets us apply different preprocessing to different column types in a single pipeline: numeric columns pass through unchanged, categorical columns get one-hot encoded. The pipeline bundles preprocessing + model into one object, so prediction is a single call.

13. Evaluation Metrics

We evaluate models using multiple metrics on a 20% held-out test set:

R^2 (coefficient of determination): Fraction of variance explained. 1.0 = perfect, 0.0 = predicts the mean. Our best model: 0.987.

MAE (Mean Absolute Error): Average absolute difference between predicted and true fuel rate, in tonnes/day. Intuitive: 'on average, how far off?'

RMSE (Root Mean Squared Error): Like MAE but penalises large errors more heavily (squares before averaging, then takes the root).

MAPE (Mean Absolute Percentage Error): MAE as a percentage of the true value. Shows relative accuracy across different ship sizes.

5-fold Cross-Validation: Instead of one train/test split, we split the data into 5 folds, train on 4, test on 1, rotate 5 times, and average. This gives a more robust estimate of model performance.

14. Simulated Annealing (SA)

Simulated annealing is an optimization algorithm inspired by the physical process of cooling molten metal (annealing). When metal cools slowly, atoms settle into a low-energy crystalline structure. If cooled too fast, they get trapped in a disordered, high-energy state.

The algorithm works by analogy:

1. Start at a random solution with a high 'temperature' T.
2. Propose a small random change (e.g., adjust speed by a small amount).
3. If the change improves the objective → always accept it.
4. If the change worsens the objective by ΔE → accept it with probability exp(−ΔE/T). This is the Metropolis acceptance rule.
5. Gradually reduce T (cooling schedule: T → T × 0.995 each step).
6. Repeat for many iterations (2000 in our code).

Early on (high T), the algorithm freely accepts bad moves — this lets it escape local minima and explore widely. As T drops, it becomes pickier and converges to the best region it found. This is fundamentally different from gradient descent, which always goes downhill and gets stuck in local minima.

```
P(accept worse move) = exp(−ΔE / T)
```

14a. Why Not Gradient Descent?

Gradient descent requires a continuous, differentiable function — it computes the slope and slides downhill. Our QUBO variables are binary (0 or 1): there is no gradient to compute. You cannot take the derivative of 'which row is the ship in at column 5.' SA works by evaluating the function at random neighbors, which only requires computing the objective value — no derivatives needed. Additionally, SA can escape local minima (via the Metropolis rule), while gradient descent cannot.

14b. Cooling Schedule

The cooling schedule controls how fast the temperature drops. Too fast → the algorithm freezes before exploring enough (gets trapped). Too slow → it wastes time accepting random moves. Our schedule: $T(n+1) = T(n) \times 0.995$, starting from $T_0 = 5.0$. After 2000 iterations, $T \approx 5 \times 0.995^{2000} \approx 0.00002$ — effectively zero, so only improvements are accepted at the end.

15. Grid Search (Brute Force Baseline)

The simplest optimization: evaluate the objective at 1000 evenly spaced points across the feasible range and pick the minimum. For a 1D problem (speed), this is fast and guaranteed to find the optimum within the grid resolution. We use it to verify that SA and QUBO find the correct answer.

For route optimization, brute force means evaluating all possible paths — which grows exponentially (rows^cols). For a 5×7 grid that's $5^7 = 78{,}125$ paths. For 11×25 it's $11^{25} \approx 10^{26}$ — impossible. This is why we need smarter methods (SA, QUBO, or dynamic programming).

16. Dynamic Programming (DP) — Exact Solver

Dynamic programming solves the routing problem exactly by exploiting its optimal substructure: the best path to cell (column c, row r) must arrive from the best path to one of the adjacent cells at column c−1.

Algorithm:

1. dp[0][start_row] = cost of the starting cell. All other dp[0][r] = ∞.
2. For each column c from 1 to n_cols−1:
   For each row r:
   dp[c][r] = min over valid predecessors (r−1, r, r+1) of:
   dp[c−1][predecessor] + cost[r][c] + diagonal surcharge
3. The answer is dp[last_col][end_row].
4. Trace back through the 'back' pointers to recover the path.

Complexity: O(rows × cols) — runs instantly on any grid. We use it to validate the QUBO answer: if QUBO's fuel is within ~1% of DP's fuel, the QUBO is working correctly.

```
dp[c][r] = min(dp[c-1][r-1], dp[c-1][r], dp[c-1][r+1]) + cost[r][c]
```

17. QUBO (Quadratic Unconstrained Binary Optimization)

QUBO is a mathematical formulation where you minimize a quadratic polynomial over binary (0/1) variables:

minimize $H = \Sigma_i q_i \cdot x_i + \Sigma_{ij} q_{ij} \cdot x_i \cdot x_j$
where each $x_i \in \{0, 1\}$

The word 'unconstrained' is misleading — constraints are not absent, they are encoded as penalty terms added to the objective. If a constraint is violated, the penalty makes the energy so large that the solution is never optimal.

QUBO is important because:

1. It is mathematically equivalent to the Ising model from physics.
2. D-Wave quantum annealers solve QUBO/Ising problems natively.
3. Many combinatorial optimization problems (routing, scheduling, partitioning) can be formulated as QUBOs.

The academic foundation is Lucas (2014), 'Ising formulations of many NP problems', which provides recipes for encoding standard problems as QUBOs.

$$H = \Sigma\ q_i \cdot x_i \ + \ \Sigma\ q_{ij} \cdot x_i \cdot x_j \quad \text{where } x_i \in \{0, 1\}$$

17a. The Ising Model Connection

The Ising model (1920s physics) describes a system of magnetic spins, each pointing up (+1) or down (−1). The system's energy depends on how neighboring spins interact. QUBO (variables 0/1) and Ising (variables −1/+1) are related by a simple substitution: $s_i = 2x_i - 1$. Any QUBO can be converted to an Ising problem and vice versa. Quantum annealers physically implement the Ising model — qubits are the spins, and programmable couplers set the interaction strengths. This is why expressing a problem as QUBO makes it directly runnable on quantum hardware.

17b. One-Hot Encoding (in QUBO)

When a decision is 'pick one from K options' (e.g., which of 40 speed levels, or which row at column 5), we create K binary variables $x_1 ... xK$, one per option. Exactly one must be 1. This is enforced by adding a penalty:

$$P \times (x_1 + x_2 + ... + xK - 1)^2$$

If exactly one $x_i = 1$, the sum is 0 (no penalty). If zero or two+ are set, the squared term is positive and the penalty P makes the energy blow up. P must be large enough that violating the constraint is never worth the fuel savings from cheating.

17c. Constraints as Penalties

QUBO has no 'subject to' section — everything must be in the single energy function. Constraints are converted to penalty terms:

- One-hot (pick exactly one): $P \times (\Sigma x_i - 1)^2$
- Deadline (forbid slow speeds): $P \times x_i$ for each infeasible i
- No teleporting (consecutive rows differ by ≤ 1): $P \times x_i \cdot x_j$ for illegal pairs
- Fixed endpoints (start/end port): $P \times x_i$ for each wrong-row variable at the first/last column

The penalty weight P is chosen as 10 × max(cell_cost) — large enough to dominate any fuel savings from constraint violation, but not so large that it drowns out the objective's fine structure.

17d. Diagonal Surcharge

When the ship changes row between consecutive columns (a diagonal move), it covers extra distance compared to going straight. We add a 10% surcharge on the destination cell's fuel cost for diagonal moves. This discourages unnecessary zigzagging while still allowing detours when storms make them worthwhile. In the QUBO, this is a quadratic term: surcharge × cost[r2,c+1] × x[c,r] × x[c+1,r2] for adjacent but different rows r, r2.

18. PyQUBO

PyQUBO is a Python library that lets you build QUBO Hamiltonians symbolically. Instead of manually computing the quadratic coefficients, you write:

```
x = Binary('x_0')
H = 3.5 * x + penalty * (x + y - 1)**2
```

PyQUBO expands, simplifies, and compiles this symbolic expression into the raw QUBO dictionary {(var_i, var_j): coefficient} that solvers need. This is far less error-prone than building the QUBO matrix by hand.

19. D-Wave neal (Simulated Annealing Sampler)

neal is D-Wave's open-source classical simulated annealing sampler. It takes a QUBO dictionary and finds low-energy binary assignments by flipping bits according to the Metropolis rule with a cooling schedule.

Key parameters:

- num_reads: how many independent annealing runs to perform (we use 500–4000 depending on grid size). The best result across all reads is returned.
- num_sweeps: how many bit-flip attempts per read (we use 1000–5000). More sweeps =

slower cooling = better quality but slower.

The critical point: the QUBO dict you pass to neal.SimulatedAnnealingSampler() is the exact same object you would pass to dwave.system.DWaveSampler() on a real quantum annealer. One line of code change. This is why we use the QUBO formulation — hardware portability.

20. Quantum Annealing (the Hardware Goal)

Quantum annealing is a physical process where a quantum computer finds the lowest-energy state of an Ising/QUBO problem using quantum mechanical effects:

1. The system starts in a superposition of all possible binary states.
2. The QUBO energy landscape is gradually 'turned on' (the annealing).
3. Quantum tunneling allows the system to pass through energy barriers that classical SA would have to climb over.
4. The system settles into a low-energy (near-optimal) state.

D-Wave's machines (Advantage, 5000+ qubits) implement this physically. Our project uses classical neal as a stand-in, but the QUBO formulation is designed to be hardware-portable: the same code runs on real quantum hardware by changing one line (the sampler).

21. Weather Routing as a Grid Problem

The ocean region between two ports is discretized into a grid of n_rows × n_cols cells. The ship starts at a port on the west side and must reach a port on the east side, advancing one column per stage (west → east). At each stage, it can stay in the same row or move one row north/south (no teleporting across multiple rows).

Each cell has a wave height (from the weather field) and a corresponding fuel cost (computed by the Phase 1 ML model). The ship's goal: find the path from start to end that minimizes total fuel. This may mean detouring north or south to avoid a storm, even though the detour is longer.

This 'monotone grid path' structure is what makes the problem naturally fit a QUBO: binary variable per cell, one-hot constraint per column, adjacency constraints between columns.

22. Storm Modelling (Gaussian Blobs)

Storms are modelled as 2D Gaussian functions on the grid. Each storm has:

- Center position (row, column)
- Amplitude (peak wave height addition, 4–7 m)
- Spread (row-wise and column-wise, controlling storm size)

The wave height at any cell is the calm-sea baseline (1 m) plus the sum of contributions from all storms, clipped to a maximum of 8 m. This creates a realistic-looking weather field where storms are localized high-wave regions that the routing algorithm should avoid.

$$waves[r,c] = 1.0 + \Sigma\ amp \times \exp(-((r-rc)^2/2\sigma r^2 + (c-cc)^2/2\sigma c^2))$$

23. ML-Driven Cell Costs

Rather than using a simple formula for cell fuel costs, we use the trained Gradient Boosting model from Phase 1. For each cell, we construct a full scenario (ship type, displacement, speed, that cell's wave height and corresponding wind) and ask the model to predict the fuel rate. This means the routing optimization benefits from the model's learned understanding of how waves, wind, ship type, and speed interact — not just a linear penalty for high waves.

Cell fuel cost = predicted_rate(t/day) × time_to_cross_cell(days)

24. Project Pipeline Overview

The project follows a phased pipeline:

Phase 1 — Data + ML + Classical Optimization:
config.py → generate_data.py → train_model.py → optimizer.py → demo.py

- Generate synthetic ship data from physics
- Train ML models to predict fuel rate
- Optimize speed via SA and grid search

Phase 2 — Quantum-Inspired QUBO:
qubo_speed.py → qubo_route.py

- Reformulate speed optimization as a QUBO (proof of concept)
- Reformulate weather routing as a QUBO (the headline result)
- Solve with neal (classical SA on QUBO), validate against exact DP

Phase 3 (planned) — Dashboard:
Streamlit web UI for interactive exploration

Each phase builds on the previous: Phase 2's QUBO routing uses Phase 1's ML model to compute cell costs. The separation into phases mirrors a real development process — validate each component before building on it.

25. Fuel Savings: Two Independent Levers

The project demonstrates two distinct ways to save fuel:

1. Speed optimization: Slow down to the economic speed or the minimum speed that meets the deadline. Savings: 29–68% depending on schedule flexibility. This is the bigger lever.

2. Route optimization: Detour around storms instead of plowing through them. Savings: 3–10% per leg depending on storm severity and placement. This is the 'smarter' lever — it requires weather data and the QUBO.

In practice, both would be used together: the fleet operator picks the optimal speed AND the optimal route for each voyage.